Agent Pre-Search Checklist

Project: Ghostfolio – AI agent for portfolio/investment domain
Framework: LangGraph (multi-agent)
Reference: AI conversation that led to this document.

Phase 1: Define Your Constraints

1. Domain Selection

Item	Answer
Which domain	Finance (portfolio / investment tracking).
Specific use cases	• `portfolio_analysis(account_id)` → holdings, allocation, performance • `transaction_categorize(transactions[])` → categories, patterns • `tax_estimate(income, deductions)` → estimated liability • `compliance_check(transaction, regulations[])` → violations, warnings • `market_data(symbols[], metrics[])` → current data
Verification requirements	• Domain-specific checks before returning responses (allocation sums, no invalid negatives, allowed symbols/currencies, compliance rules, tax consistency). • Human-in-the-loop for any action that moves money (see §3).
Data sources	**Existing (use as-is):** DataProviderService (quotes, historical, dividends, asset profile, search), ExchangeRateDataService, PortfolioService, OrderService, MarketDataService. **Gaps to address:** live fundamentals (P/E, etc.), news, compliance/regulatory lists, tax rates/rules, transaction categorization taxonomy (rules or external API).

2. Scale & Performance

Item	Answer
Expected query volume	Expressed **per user**: e.g. queries per user per day (or per month). Design limits and capacity from “N users × Q queries/user/day” rather than raw RPS only. **Rate limit:** throttle “LLM whales” (users who burn disproportionate LLM

Item	Answer
	quota) unless they're on a top-tier / higher-paying plan that grants higher limits. Define tiers (e.g. free/Basic: X queries/day; Premium: Y; enterprise/custom: Z or unlimited).
Acceptable latency	**On par with industry standards for AI apps:** time-to-first-token (TTFT) <~2s for perceived responsiveness; full response latency depends on tool round-trips (multi-step agent flows will be longer—give clear feedback, e.g. "Checking portfolio…"). Target sub-2s TTFT for simple answers; acceptable longer for multi-tool flows if the UI shows progress.
Concurrent user requirements	**Horizontal scaling:** the app scales horizontally; add more instances as load grows. No single-node bottleneck by design—spin up more instances to handle concurrent users.
Cost constraints for LLM calls	Managed via **per-user rate limits** (see query volume). High-tier or paying users get higher limits; default tier keeps abuse in check. Track cost per user/session for visibility and tier tuning.

Benchmark notes (non-AI apps; proxy for usage scale and infra):

- **Robinhood (retail investing):** ~27M funded customers (2025). Brokerage at 100k→750k→2M+ req/s peak. MAU ~12.5M (2022). Use to size "total platform"; agent traffic is a fraction (per-user query caps keep peak RPS predictable).
- **Monarch Money (personal finance):** ~$30M ARR (2024), strong growth. Smaller scale than Robinhood; good "serious but not massive" benchmark for per-user and total load.
- **Plaid (finance API):** 300+ req/s sustained; P95 <2s; 99.5–99.9% uptime—align agent endpoint with similar SLA where possible.
- **AI latency (industry):** 2024 norms: sub-1s for "instant" chat; ~2s TTFT as psychological threshold; full agent response longer when tools are involved—streaming and progress cues matter.

3. Reliability Requirements

Item	Answer
Cost of a wrong answer?	High in finance (incorrect allocation/tax/compliance can have real impact). Assumption: treat as high-severity domain.
Non-negotiable verification?	(1) Domain-specific verification layer before returning answers. (2) Human approval before any money-moving action.

Item	Answer
Human-in-the-loop requirements?	**Required** for anything that actually moves money (e.g. execute_trade). Single approval point on the action agent's execute path.
Audit/compliance needs?	Conversation history persisted (Postgres); relational storage supports queryable, auditable history. **To research:** what data must be retained for audits (e.g. thread id, user id, message role/content, tool calls/results, timestamps, approval events, model version); retention period; regulatory expectations by jurisdiction.

4. Team & Skill Constraints

Item	Answer
Familiarity with agent frameworks?	Some prior experimentation with LangChain; chose LangGraph for explicit flow and state.
Experience with domain?	Working in Ghostfolio codebase (portfolio, orders, market data, data providers).
Comfort with eval/testing frameworks?	Current stack (Jest + Nx + NestJS testing) considered sufficient for unit/integration tests; no additional test framework planned for agent work.

Phase 2: Architecture Discovery

5. Agent Framework Selection

Item	Answer
LangChain vs LangGraph vs CrewAI vs custom?	**LangGraph.**
Single vs multi-agent?	**Multi-agent.** Specialist agents compile info (portfolio, market, compliance, tax) and pass **summaries** to an action agent; saves context and gives a clear place for human verification.
State management?	**Postgres (relational)** for conversation history (threads + messages). **Redis** already in stack for cache; optional for hot/checkpoint state.

Item	Answer
Tool integration complexity?	Five tools with schema + execute; LangGraph tool node runs them. Existing app services (DataProvider, Portfolio, etc.) used inside tool `execute`.

6. LLM Selection

Item	Answer
GPT-5 vs Claude vs open source?	**Lock in GPT-5** (via OpenRouter or direct). Single default model for reliability and fewer failure points (e.g. avoiding churn from providers changing APIs).
Function calling support?	Required; LangGraph + tool-calling loop depend on it.
Context window needs?	Multi-agent design keeps action-agent context smaller (summaries only). Set to model's native window; no extra constraint.
Cost per query acceptable?	Tracked and optimized; see §8 (observability). Latency and cost are foremost; use hosted tooling to monitor.

7. Tool Design

Item	Answer
Tools needed?	`portfolio_analysis`, `transaction_categorize`, `tax_estimate`, `compliance_check`, `market_data` (at least 3 functional for MVP; all five planned).
External API dependencies?	Existing data providers (Yahoo, Alpha Vantage, etc.) for market/portfolio. Possible new: fundamentals, news, compliance lists, tax data (TBD).
Mock vs real for development?	Use existing services; mock in Jest for unit tests. TBD: dedicated mock data sets for agent eval if needed.
Error handling per tool?	Wrap each tool `execute` in try/catch; return structured error (e.g. `{ error: true, message }`) into state so the model can respond gracefully; no unhandled crashes.

8. Observability Strategy

Item	Answer
LangSmith vs Braintrust vs other?	**Hosted where possible**—"stand on the shoulders of giants." Use a hosted observability/tracing product (e.g. LangSmith, Braintrust, Helicone) for traces, logs, and cost.
Metrics that matter most?	**Latency and cost are foremost.** Then: tool success rate, human override rate, error rate.
Real-time monitoring needs?	Use hosted solution for real-time dashboards and alerts (latency, errors, cost spikes).
Cost tracking requirements?	Track cost per query/session/user; required for rate limits and tier tuning. Hosted tooling preferred.

9. Eval Approach

Item	Answer
How to measure correctness?	**Golden test cases** with obvious, common-sense expected responses. Automated checks (output shape, domain checks). Failures on these cases → **flag for human review.**
Ground truth data sources?	A **small set of test cases** with known-good answers (common-sense responses). Expand over time (portfolio snapshots, hand-labeled categories, tax/compliance outcomes as needed).
Automated vs human evaluation?	Automated: run golden set in CI; failures trigger human review. Jest for unit/integration; human review for flagged agent outputs.
CI integration for eval runs?	Yes. Run golden test cases in CI; failures block or alert and route to human review.

10. Verification Design

Item	Answer
Claims that must be verified?	Numeric consistency (allocations, totals), no invalid negatives, symbol/currency validity, compliance rules, tax math consistency.

Item	Answer
Fact-checking data sources?	App's own data (portfolio, orders, market data); compliance/tax from rules or external sources TBD.
Confidence thresholds?	When confidence is **too low**: **withhold** and direct user to a trusted human advisor (e.g. "I'm not sure—for this kind of question, please consult a trusted human advisor"). No guess when uncertain.
Escalation triggers?	Low confidence → "find a trusted human advisor for this." Tool failure, verification failure, or human-required step (e.g. money-moving) → clear message + escalation path. Log these for audit and product review.

Phase 3: Post-Stack Refinement

11. Failure Mode Analysis

Item	Answer
When tools fail?	Return structured error into state; agent can explain or retry. No crash.
Ambiguous queries?	TBD: choose clarify (ask user) vs best-effort answer with caveat; document in runbook.
Rate limiting and fallbacks?	Per-user rate limits (tiered); TBD: provider rate limits (backoff/queue) and fallback data sources or cached responses.
Graceful degradation?	TBD: e.g. respond without a failed tool's data and state the gap ("I couldn't fetch X; here's what I have…").

12. Security Considerations

Item	Answer
Prompt injection prevention?	TBD: e.g. input sanitization, prompt structure, output validation; define before launch.
Data leakage risks?	Strict user/tenant isolation: state and context scoped to user/session only. TBD: audit for cross-tenant edge cases.

Item	Answer
API key management?	Existing: OpenRouter (and others) via PropertyService. Agent tools use same pattern. TBD: any new keys for new data sources (store in PropertyService or env).
Audit logging requirements?	Conversation history in Postgres supports audit. TBD: exactly what to log (e.g. tool calls, approvals, model version) and retention; align with audit research in §3.

13. Testing Strategy

Item	Answer
Unit tests for tools?	Yes; Jest, mock dependencies (DB, Redis, data providers), assert on tool output shape and error paths.
Integration tests for agent flows?	Yes; NestJS testing module, mock LLM/checkpoint, test graph state and tool invocation. Optional: supertest for HTTP agent endpoint.
Adversarial testing?	TBD: e.g. prompt injection and jailbreak-style tests; add if security review requires it.
Regression testing?	Jest in CI. TBD: add eval/regression suite for agent outputs (golden set covers baseline).

14. Open Source Planning

Item	Answer
What will you release?	TBD: e.g. agent + tools only, or full LangGraph integration; scope of docs and examples.
Licensing?	Ghostfolio is AGPL-3.0; TBD: how agent code is licensed and whether new dependencies affect that.
Documentation requirements?	TBD: minimal (README + checklist) vs full (architecture, runbooks, API).
Community engagement plan?	TBD: if/when to announce, issue templates, contribution guidelines.

15. Deployment & Operations

Item	Answer
Hosting approach?	TBD: same as existing API (single deploy) unless scale or isolation demands a separate agent service.
CI/CD for agent updates?	TBD: same pipeline as app by default; separate only if release cadence differs.
Monitoring and alerting?	Hosted observability (§8) for latency, errors, cost; TBD: alert thresholds and on-call runbook.
Rollback strategy?	TBD: e.g. feature flag or versioned agent config; same as app rollback if single deploy.

16. Iteration Planning

Item	Answer
How to collect user feedback?	TBD: in-app (thumbs, “Report”) and/or support channels; log with thread id for review.
Eval-driven improvement cycle?	TBD: e.g. weekly run of golden set, review flagged failures, prioritize fixes and new cases.
Feature prioritization?	TBD: align with product roadmap; agent capabilities follow tool and data source readiness.
Long-term maintenance plan?	TBD: owner, dependency upgrades (LangGraph, SDK, GPT-5 API), and eval regression as part of release process.

Summary

Decided: Domain (finance), five tools, LangGraph, multi-agent, Postgres state, GPT-5, hosted observability (latency + cost first), golden-set eval with human review on failure, low-confidence → “find a trusted human advisor,” Jest testing, rate limits and horizontal scaling.

TBD / To research: Audit data and retention (§3, §12), ambiguous-query handling, provider rate limits and fallbacks, graceful degradation, prompt injection and data-leak controls,

adversarial/regression testing, open source scope and licensing, deployment and CI/CD details, feedback and iteration process.

Chat History (reconstructed)

User: Ignoring the Readme, what do you think this repo is trying to accomplish?

Assistant: [Explored codebase: package.json, prisma schema, API structure, portfolio/order/import controllers.] Summarized: self-hosted personal investment and portfolio tracker—accounts, orders (activities), symbol profiles, market data; portfolio analytics (holdings, performance, dividends, reports); import/export; multi-user and sharing; subscription; watchlist, tags, benchmarks; auth (JWT, OAuth, OIDC, WebAuthn); data providers (e.g. Yahoo Finance). Stack: NestJS API, Angular client, Prisma/Postgres, Nx monorepo.

User: Anything wired to the AI endpoint yet?

Assistant: [Checked AI controller, service, client.] Wired: GET /api/v1/ai/prompt/:mode builds prompt (portfolio table or analysis template) and returns it; Analysis page calls it and copies prompt to clipboard with “Open Duck.ai” nudge. Not wired: AiService.generateText() (OpenRouter + AI SDK) exists but is never called—no in-app LLM call.

User: I see so the code base currently reads in AI prompts but doesn't process or actually call any agents?

Assistant: Clarified: app *generates* prompts (from portfolio data), doesn’t “read in” prompts. No agent/LLM is called in the normal flow; only use is copy-to-clipboard for external sites. generateText() is dead code.

User: Then lets do a pro con for langchain and langgraph

Assistant: [Provided pro/con tables for LangChain vs LangGraph: ecosystem vs complexity, explicit flow/state, when each fits. Recommendation: LangGraph for flow/state/human-in-loop; LangChain for chains+tools+RAG.]

User: I was thinking things like Langchain. Maybe something else since I've already experimented with that once

Assistant: [Outlined options: double down on Vercel AI SDK (already in repo), LangGraph, CrewAI, hand-rolled. Recommendation: AI SDK for “one agent + tools” or LangGraph for explicit graphs; CrewAI for multi-agent crews.]

User: these are 5 tools that need to be implemented. Do you think this rules out any frameworks or hand rolling? portfolio_analysis(account_id) → holdings, allocation, performance transaction_categorize(transactions[]) → categories, patterns tax_estimate(income, deductions) → estimated liability compliance_check(transaction, regulations[]) → violations, warnings market_data(symbols[], metrics[]) → current data

Assistant: No. Fits single agent + tools (AI SDK or hand-rolled). LangGraph optional for explicit flow. CrewAI poor fit (multi-agent delegation). Not ruled out: AI SDK, hand-roll, LangGraph.

User: Alright langgraph it is. I'm told "Conversation history maintained across turns" is thee anything currently in the architecture that can support this or should I add redis/a relational db?

Assistant: [Checked Redis/cache and Prisma schema.] Nothing currently stores conversation/chat. Redis already in stack (cache, portfolio snapshots). No conversation models in Postgres. Recommendation: Redis for thread state (existing infra) or add Postgres tables (AgentThread, AgentMessage) for durable queryable history; add relational when you need audit/list threads.

User: relational or nosql? I've been told that if you don't need to order or count you should do nosql

Assistant: You do need order (message sequence) and count (messages per thread, threads per user). That heuristic points to relational. Recommendation: Postgres for threads/messages; Redis optional for hot state.

User: Agent Components [Reasoning Engine, Tool Registry, Memory, Orchestrator, Verification Layer, Output Formatter] — I need to have these components. Would that necessitate the extra complexity of langgraph?

Assistant: No. All six can be implemented with a simpler stack (e.g. Vercel AI SDK): reasoning = LLM + prompt; tool registry = tools object; memory = message array + DB; orchestrator = tool loop with maxSteps; verification = post-loop check; output = structured output. LangGraph worth it when you want explicit graph, state persistence, or human-in-loop branching.

User: Tool calls execute successfully and return structured results; Agent synthesizes tool results into coherent responses; Basic error handling — this is all part of just general functionality for a tool right? Shouldn't be too hard to configure in langgraph I suppose.

Assistant: Yes. Standard tool-calling loop; LangGraph handles it. Wrap tool execute in try/catch, return structured error into state; model can explain or retry.

User: Then lets do a pro con for langchain and langgraph

Assistant: [Already provided earlier in conversation.]

User: what's a domain-specific verification check

Assistant: A check you run *after* the agent answers, using *your domain's* rules (finance, compliance, tax), not generic code. Examples: allocation percentages sum to 100%; no invalid negatives; symbol/currency validity; compliance rules; tax math consistency. Verification layer = run these before returning the response.

User: I'd probably need tools to access datasources for some of these functions. Are there some common use cases that the app doesn't support?

Assistant: [Checked DataProvider interface and usage.] Supported: quotes, historical, dividends, asset profile, search, sectors/countries (enhancers), FX. Not supported: live fundamentals (P/E, etc.), news, compliance/regulatory lists, tax rates/rules, transaction categorization taxonomy. Tools can use existing services for portfolio_analysis and market_data; add sources or internal data for the rest.

User: I'd probably need human verification for anything that actually moves money. single or multiagent? I suppose having agents be individually available to compile info for other agents could save on context.

Assistant: Human verification: same in single or multi-agent—gate execution (tool needsApproval or graph node "human_review" before execute). Multi-agent: specialists (research, compliance) compile summaries; action agent gets summaries only → smaller context; single approval point on action agent. Recommendation: multi-agent for context saving; human verification on the path that moves money.

User: Based on what we discussed create a document that addresses this checklist. If there's anything we haven't addressed ask me questions.

Assistant: [Created docs/agent-pre-search-checklist.md with full Pre-Search Checklist, filled from discussion, and listed 11 open questions at end.]

User: I'll have to research more about what data needs to be maintained for audits. Is there any current estimates for how much like robinhood or monarch does for the scale/performance stuff? They dont' have AI but could be a metric for usage

Assistant: [Web search: Robinhood ~27M funded, 100k→2M+ req/s; Monarch ~$30M ARR, growth; Plaid 300+ req/s, P95 <2s.] Updated checklist: benchmark notes (Robinhood, Monarch, Plaid, AI latency); audit = “To research” (what to retain, retention, jurisdiction).

User: Latency... should be on par with industry standards for AI apps. query volume should be expressed in terms of per user. Probably rate limit people if they're an LLM whale unless they're paying top dollar. Concurrent user... I mean this is an app that'll scale horizontally well so just spin up more instances. Add all that and address pt 2

Assistant: [Updated §2: latency = industry standard for AI (TTFT <~2s); query volume per user + rate limit whales unless top tier; concurrent = horizontal scaling; cost via rate limits and tracking.]

User: Alright lets run down the rest of the open stuff. Like 4 "opens" at a time ask me questions until we nail the rest of the document

Assistant: [Asked Batch 1: (1) LLM—single default or configurable? context/cost? (2) Observability—hosted vs in-house? 2–3 key metrics? (3) Eval—automated only or human review? ground truth? (4) Verification—withhold vs best-effort vs escalate? escalation UX?]

User: Lets lock in GPT 5. For building a reliable product and having less failure pts. (like companies chaging their API). Lets use hosted where possible to stand on the shoulders of giants. Latency and cost are key questions so those are foremost. For eval I'll have a few test cases with obvious common sense responses. If the model fails those that would be a flag for human review. For verification, when confidence gets too low have it be like "not sure find a trusted human advisor for this complex shit"

Assistant: [Updated checklist §6 (GPT-5, reliability), §8 (hosted observability, latency+cost first), §9 (golden test cases, failures→human review, CI), §10 (low confidence→withhold, “find trusted human advisor”), and Summary.]

User: I'll have to research more about what data needs to be maintained for audits. Is there any current estimates for how much like robinhood or monarch does for the scale/performance stuff?

Assistant: [Same as earlier: added benchmark notes and audit research note to doc.]

User: alright clear this up

Assistant: [Condensed doc to ~80 lines: single “Decided” bullet list, short “TBD” list, condensed reference. Removed tables and long phase sections.]

User: undo that last update. I want the og format but with the open spots filled

Assistant: [Restored full Phase 1–3 checklist with all 16 section tables; filled every previously open cell with decided answers or explicit TBD + short note. Kept Summary.]

User: paste the exact entire history of this chat to the end of that markdown file

Assistant: [Appended this reconstructed chat history. Note: exact transcript not available; this is a reconstruction from context.]